



Arquitectura y despliegue

Dirigido a: **Responsables técnicos, comité de dirección y operadores que necesitan entender cómo se despliega y aísla la plataforma.**

La plataforma se opera como un **stack Docker Compose en una sola máquina** (no Kubernetes). Este manual muestra su **topología real**: un único punto de entrada (el proxy inverso **Caddy**, que termina TLS y sirve la SPA y la API en el mismo origen), una capa de **datos e infraestructura** en redes internas, y una capa de **ejecución** donde el código del usuario corre en **contenedores efímeros aislados**.

Tras la vista general, el manual desciende **plano a plano** y explica el papel de CADA servicio del compose: la superficie de entrada (Caddy), el plano de aplicación (api-server, admin-panel), el plano de ejecución (orchestrator, los pools de workers Celery y el docker-socket-proxy), el plano de datos (PostgreSQL+pgvector, Redis, MinIO, Vault), los servicios de dominio (Docling, ClamAV, Ollama, SearXNG, voz), la salida a internet controlada (egress-proxy y registry-proxy), las tres redes, la postura de seguridad de los contenedores, la pila de observabilidad (Prometheus, node-exporter, Alertmanager, cAdvisor, Grafana), el proxy firmado de la validación humana (ADR 0062) y el subsistema de copias de seguridad.

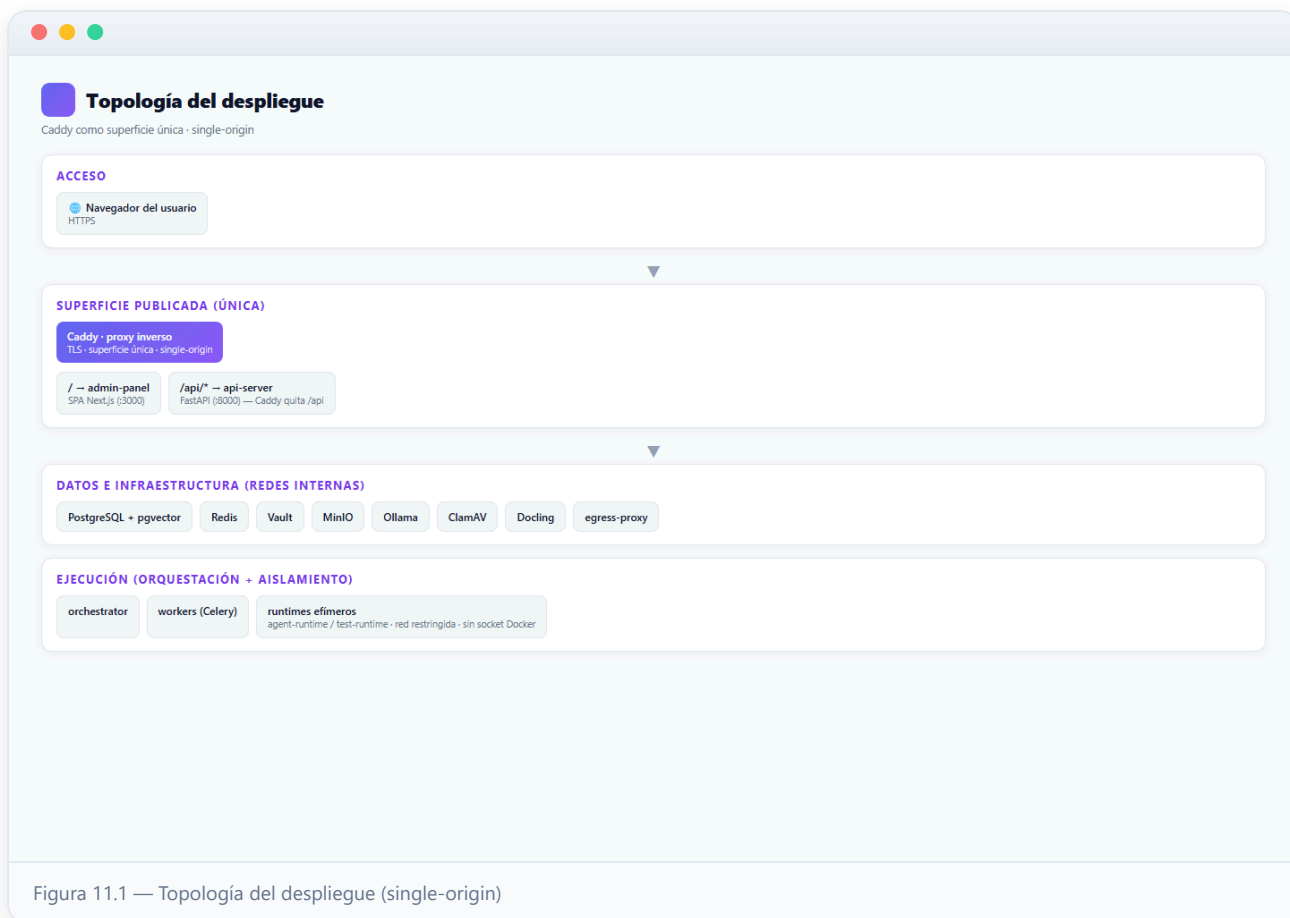
Las capturas de este manual reflejan el **despliegue en ejecución** en el momento de generarlo: las tablas de servicios muestran la imagen y el estado REALES del snapshot de docker compose ps.

Contenido

- 01** Topología del despliegue (single-origin)
- 02** Contenedores en ejecución (docker compose ps)
- 03** Aislamiento por contenedor (runtimes efímeros)
- 04** Caddy: la única superficie publicada (single-origin)
- 05** Plano de aplicación: api-server y admin-panel
- 06** Plano de ejecución: orchestrator, pools de workers y docker-socket-proxy
- 07** Plano de datos: PostgreSQL + pgvector, Redis, MinIO y Vault
- 08** Servicios de dominio: Docling, ClamAV, Ollama, SearXNG y voz
- 09** Salida a internet controlada: egress-proxy y registry-proxy
- 10** Las redes del stack: agentic-net, agentic-agents y agentic-docker
- 11** Postura de seguridad de los contenedores
- 12** Observabilidad: Prometheus, node-exporter, Alertmanager, cAdvisor y Grafana
- 13** Validación humana: review-runtime y proxy firmado (ADR 0062)
- 14** Copias de seguridad y durabilidad de los datos

1 Topología del despliegue (single-origin)

El usuario accede por **HTTPS** a una **única superficie publicada**: Caddy. Caddy sirve la SPA del panel en `/` y enruta `/api/*` al api-server (quitando el prefijo). Todo lo demás —base de datos, cache, secretos, almacenamiento, modelos— vive en **redes internas** sin puertos al host.



2 Contenedores en ejecución (docker compose ps)

Este es el **stack real en ejecución**. Cada servicio corre como un contenedor con su healthcheck; la plataforma vigila su salud (lo ves también en el **Dashboard → Salud de servicios**). Solo **Caddy** publica puertos al host: es la única superficie accesible desde fuera.

Contenedores en ejecución

Salida real de docker compose ps

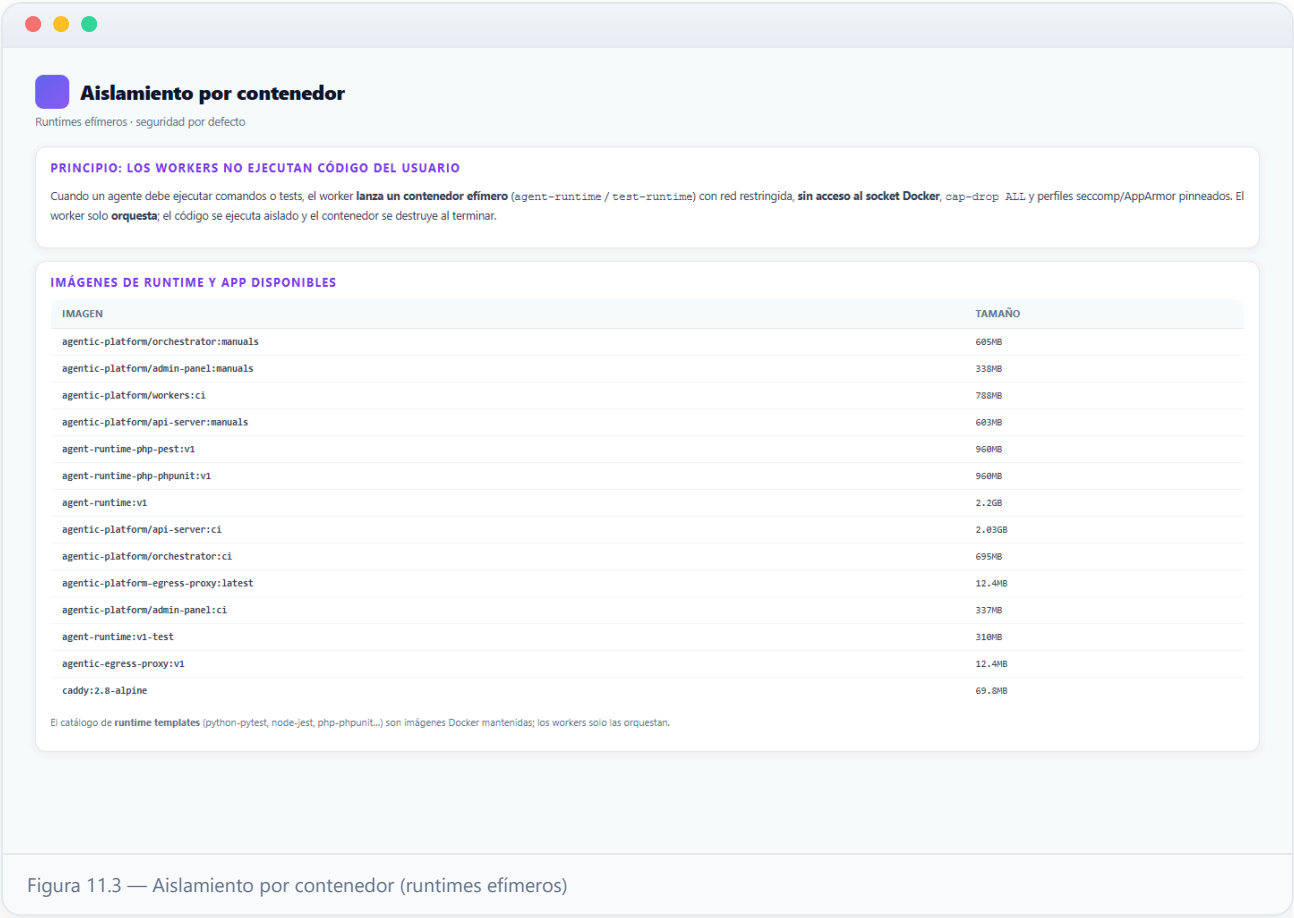
CONTENEDOR	IMAGEN	ESTADO	PUERTOS PUBLICADOS
agentic-review-demo	nginx:alpine	healthy	80/tcp
agentic-platform-admin-panel-1	agentic-platform/admin-panel:manuals	healthy	3000/tcp
agentic-platform-workers-backup-1	agentic-platform/workers:ci	healthy	8000/tcp
agentic-platform-workers-aux-1	agentic-platform/workers:ci	healthy	8000/tcp
agentic-platform-workers-1	agentic-platform/workers:ci	healthy	8000/tcp
agentic-platform-cortex-beat-1	agentic-platform/workers:ci	healthy	8000/tcp
agentic-platform-api-server-1	agentic-platform/api-server:manuals	healthy	8000/tcp
agentic-platform-orchestrator-1	agentic-platform/orchestrator:manuals	healthy	8000/tcp
agentic-platform-cadvisor-1	gcr.io/cadvisor/cadvisor:v0.49.1	healthy	8080/tcp
agentic-platform-ollama-1	ollama/ollama:v0.31.1	healthy	0.0.0.0:11434->11434/tcp, [::]:11434->11434/tcp
agentic-registry-proxy	agentic-platform/registry-proxy	healthy	8080/tcp
agentic-platform-docker-socket-proxy-1	tecnativa/docker-socket-proxy:0.3.0	healthy	2375/tcp
agentic-platform-caddy-1	caddy:2.8-alpine	healthy	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
agentic-platform-stt-1	fedirz/faster-whisper-server:latest-cpu	healthy	-
agentic-platform-tts-1	ghcr.io/remsky/kokoro-fastapi-cpu:v0.2.2	healthy	-
agentic-platform-vault-unsealer-1	hashicorp/vault:1.17	healthy	8200/tcp
agentic-platform-vault-1	hashicorp/vault:1.17	healthy	0.0.0.0:8200->8200/tcp, [::]:8200->8200/tcp
agentic-egress-proxy	agentic-platform/egress-proxy	healthy	0.0.0.0:8888->8888/tcp, [::]:8888->8888/tcp
agentic-platform-docling-serve-1	ghcr.io/docling-project/docling-serve:v1.20.0	healthy	0.0.0.0:5001->5001/tcp, [::]:5001->5001/tcp
agentic-platform-redis-1	redis:7-alpine	healthy	0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
agentic-platform-clamav-1	clamav/clamav:1.4	healthy	0.0.0.0:3310->3310/tcp, [::]:3310->3310/tcp
agentic-platform-postgres-1	pgvector/pgvector:pg16	healthy	0.0.0.0:15432->15432/tcp, [::]:15432->15432/tcp
agentic-platform-minio-1	minio/minio:RELEASE.2024-11-07T00-52-20Z	healthy	0.0.0.0:9000-9001->9000-9001/tcp, [::]:9000-9001->9000-9001/tcp
agentic-platform-grafana-1	grafana/grafana:11.2.0	healthy	0.0.0.0:3001->3000/tcp, [::]:3001->3000/tcp
agentic-platform-node-exporter-1	prom/node-exporter:v1.8.2	healthy	9100/tcp
agentic-platform-alertmanager-1	prom/alertmanager:v0.27.0	healthy	0.0.0.0:9093->9093/tcp, [::]:9093->9093/tcp
agentic-platform-prometheus-1	prom/prometheus:v2.54.1	healthy	0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp

Snapshot real de docker: compose ps (27 contenedores, capturado 2026-07-09). En este entorno de demostración algunos servicios publican puertos para depuración; en producción solo el proxy Caddy queda expuesto y el resto vive en redes internas.

Figura 11.2 — Contenedores en ejecución (docker compose ps)

3 Aislamiento por contenedor (runtimes efímeros)

El segundo principio rector del sistema: **aislamiento por contenedor**. El código del usuario y los tests NUNCA corren en el worker, sino en **runtimes efímeros** lanzados bajo demanda, con red restringida, sin socket Docker, `cap-drop ALL` y perfiles de seguridad pinneados.

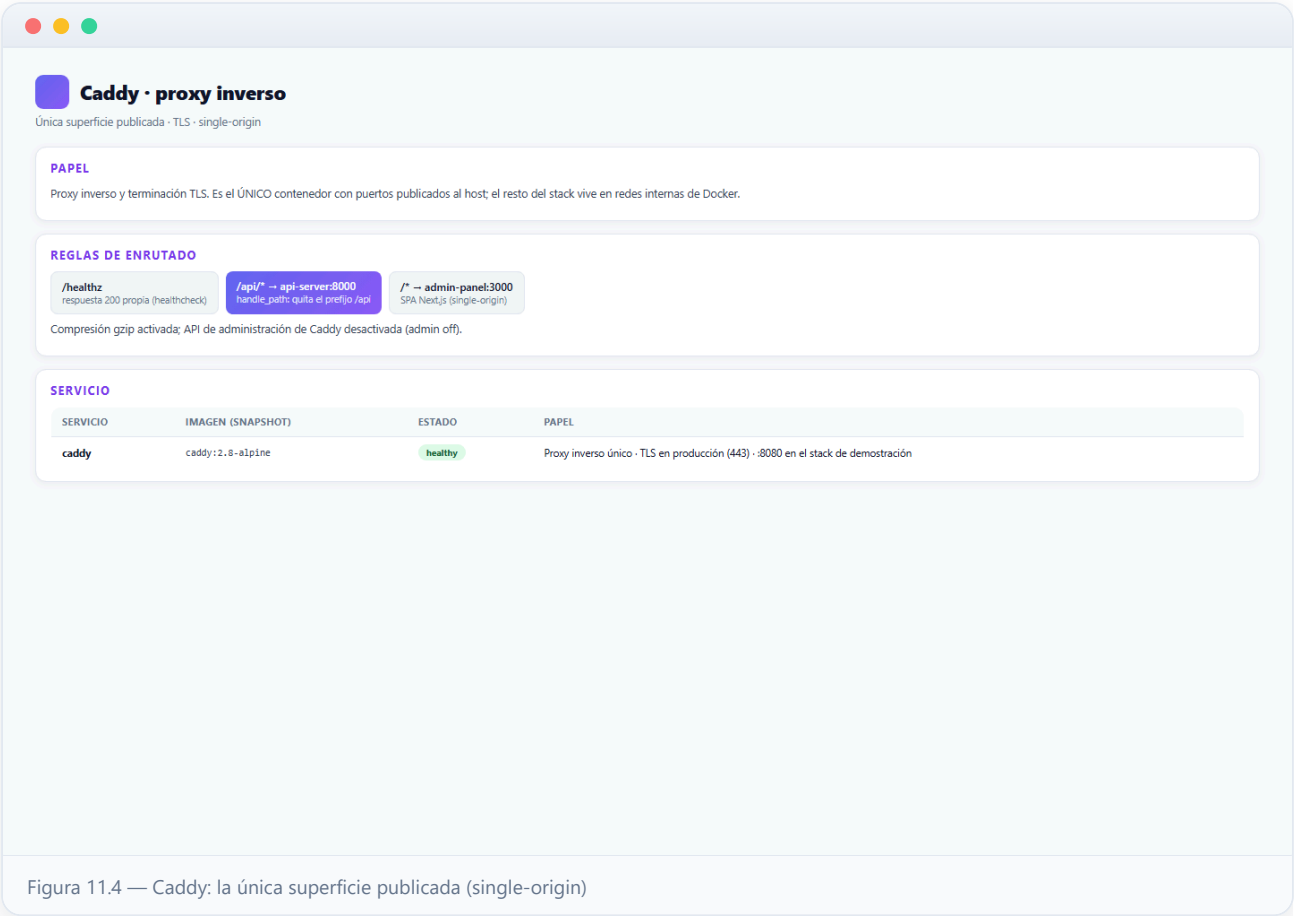


4 Caddy: la única superficie publicada (single-origin)

Caddy es el único servicio del stack que publica un puerto al host: en producción termina TLS en el 443 (el instalador genera el Caddyfile, ADR 0061); en el stack de demostración sirve HTTP plano en el 8080. Todo lo que no entra por Caddy simplemente no es alcanzable desde fuera de la máquina.

Sus reglas de enrutado son deliberadamente mínimas: `/healthz` responde un 200 propio (es el healthcheck del contenedor, sin tocar los upstreams); `/api/*` se reenvía al api-server **quitando el prefijo** (el api-server ve sus rutas reales: `/auth/login`, `/projects`, ...); y cualquier otra ruta va a la SPA del admin-panel. Las respuestas se comprimen con gzip y la API de administración de Caddy está **desactivada** (`admin off`) para reducir superficie.

El diseño **single-origin** es una decisión de seguridad y de simplicidad: la SPA se compila con `NEXT_PUBLIC_API_URL=/api`, de modo que todas las llamadas REST y WebSocket son del mismo origen — sin CORS, sin cookies de terceros, sin listas de orígenes que mantener. Cambiar de dominio solo exige tocar el Caddyfile.

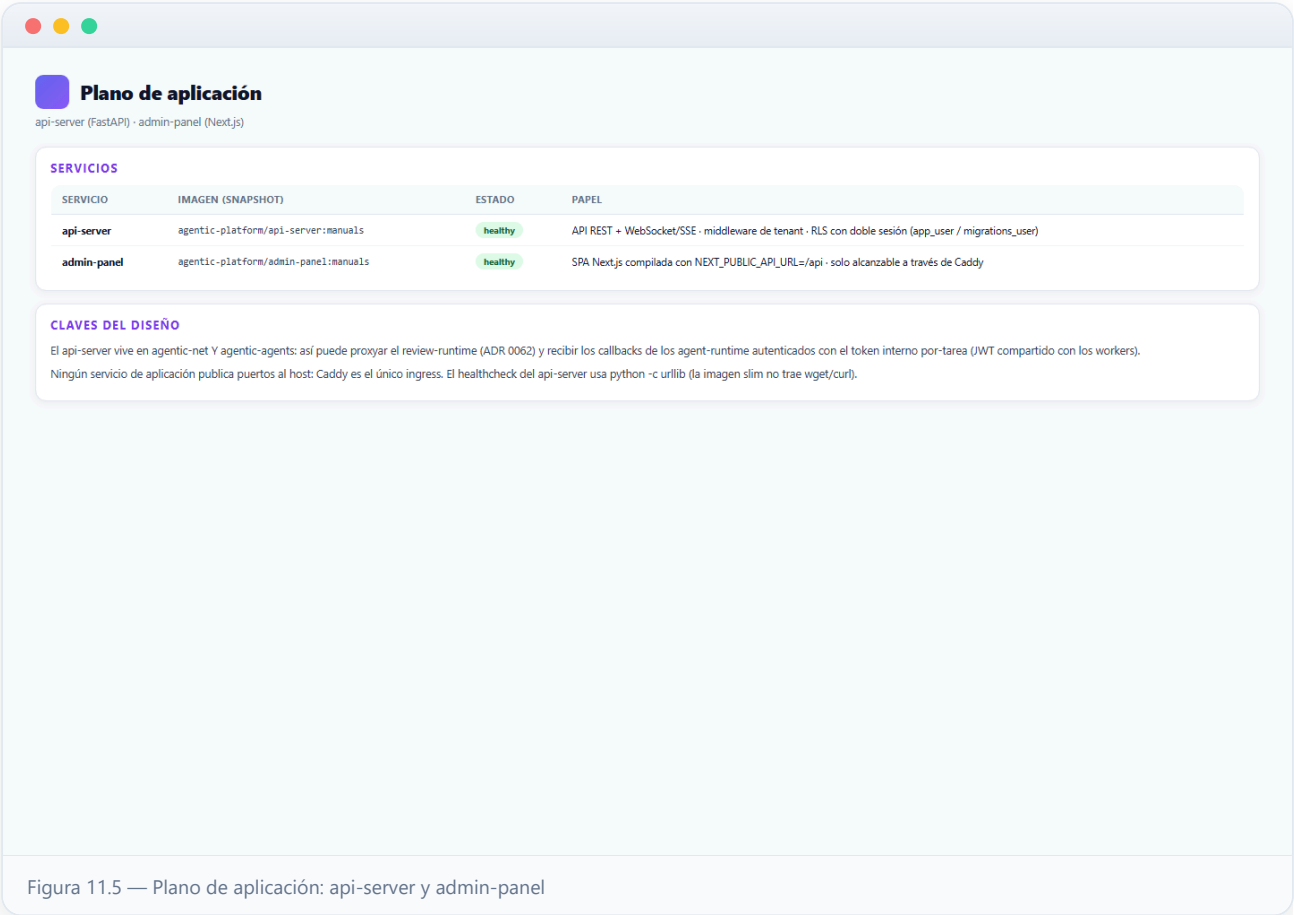


5 Plano de aplicación: api-server y admin-panel

El **api-server** (FastAPI + Uvicorn, puerto 8000 interno) es el corazón de la plataforma: expone la API REST y los canales WebSocket/SSE, aplica el middleware de tenant en cada request y habla con PostgreSQL a través de **dos sesiones** deliberadamente separadas — la sesión de aplicación (rol `app_user`, sujeto a RLS: físicamente incapaz de leer filas de otro tenant) y la sesión administrativa (rol `migrations_user`, BYPASSRLS, reservada a operaciones globales del System Admin). Arranca solo cuando PostgreSQL y Redis reportan healthy, y su propio healthcheck se hace en Python puro porque la imagen `python:3.12-slim` no trae wget/curl.

Está conectado a DOS redes: `agentic-net` (la de plataforma) y `agentic-agents` (la interna de los agentes). Esta doble pertenencia es la que le permite ejercer de **proxy firmado del review-runtime** (ADR 0062, paso posterior) y recibir los callbacks internos de los agent-runtime: para cada tarea, el worker acuña un token interno de corta vida firmado con el MISMO secreto JWT que usa el api-server, y el runtime llama de vuelta a `http://api-server:8000` con ese token.

El **admin-panel** es la SPA Next.js compilada con `NEXT_PUBLIC_API_URL=/api`: no conoce el dominio de la instalación, todas sus llamadas son relativas al origen y viajan a través de Caddy. Sirve en el 3000 interno y no publica puertos; depende del api-server healthy para arrancar.



Plano de ejecución: orchestrator, pools de workers y docker-socket-proxy

El **orchestrator** vigila el DAG de cada plan y despacha las tareas listas a las colas de Celery (expone su `/healthz` en el 8002 interno). Los **workers** se organizan en pools especializados por colas:

- **workers** (colas `default`, `ingestion`, `test`, `review`, concurrencia 2): el pool principal. NUNCA ejecuta código del usuario — lanza contenedores **agent-runtime efímeros** a través del docker-socket-proxy y orquesta su ciclo de vida completo (worktree, presupuesto, timeout, veredicto).
- **workers-aux** (colas `test`, `review`): pool auxiliar que evita la auto-inanición — un run de agente que espera el resultado de un `stack_exec` (ADR 0093) no puede ocupar el mismo slot que ese comando necesita para ejecutarse.
- **workers-backup** (cola `privileged`, concurrencia 1): el único pool con privilegios de root, dedicado en exclusiva a backups y restores (necesita leer y escribir los datos internos de Redis y Vault, que son 0700). No ejecuta runs de agentes.
- **cortex-beat**: el scheduler (Celery beat) que agenda los bucles de fondo del córtex; solo encola por horario — y con el kill-switch de autonomía apagado (el valor por defecto) sus ticks son no-op.

El **docker-socket-proxy** (ADR 0060) es la pieza que permite a los workers lanzar contenedores sin entregarles el socket de Docker: monta `/var/run/docker.sock` en **solo lectura** y expone una API mínima — solo contenedores, imágenes, redes, POST y exec están habilitados; volúmenes, swarm, secretos, system e info están denegados. Vive en una red interna dedicada (`agentic-docker`) a la que solo se conectan los workers; el agent-runtime jamás lo alcanza.

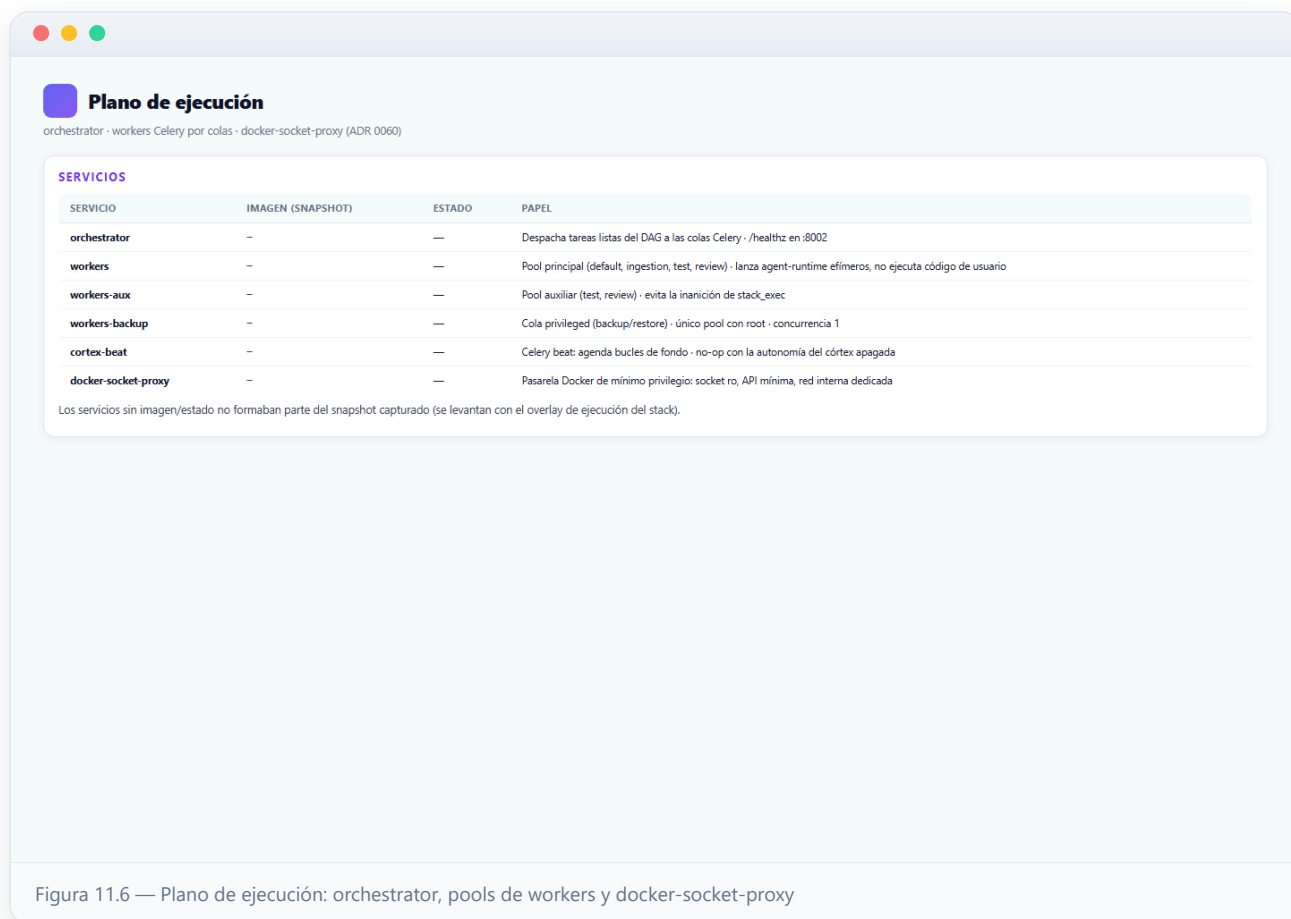


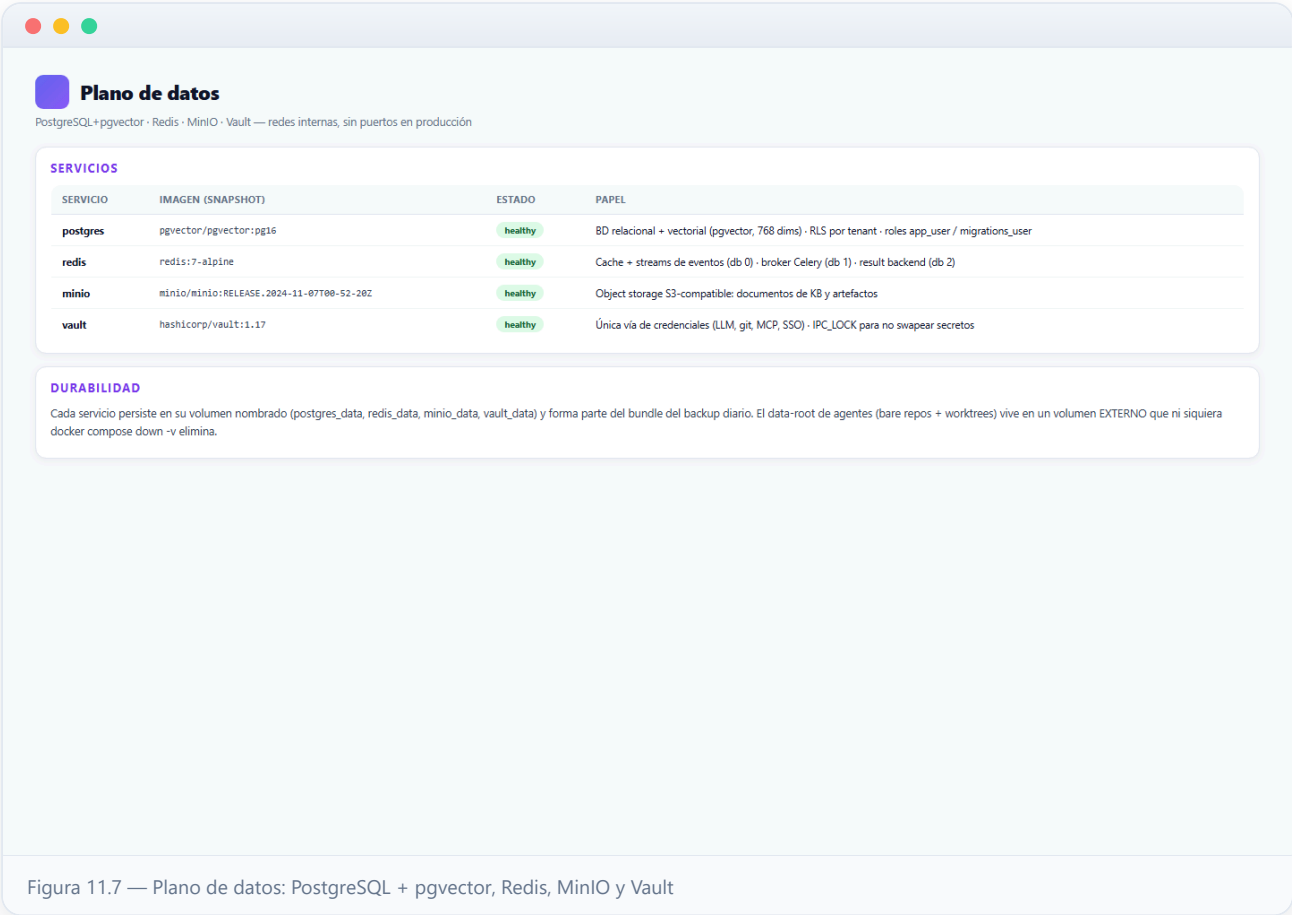
Figura 11.6 — Plano de ejecución: orchestrator, pools de workers y docker-socket-proxy

7 Plano de datos: PostgreSQL + pgvector, Redis, MinIO y Vault

PostgreSQL 16 es a la vez la base relacional y la vectorial: la extensión **pgvector** vive en la misma base de datos (embeddings de 768 dimensiones para memoria y RAG), lo que evita operar un segundo motor. El aislamiento multi-tenant se aplica *en el propio motor*: cada tabla lleva `tenant_id` y Row-Level Security activado, con dos roles de conexión — `app_user` (sujeto a RLS) y `migrations_user` (BYPASSRLS, para migraciones Alembic y administración global).

Redis 7 cumple tres papeles sobre bases lógicas separadas: cache y streams de eventos en tiempo real (db 0, los que alimentan el despacho y los WebSocket), broker de Celery (db 1) y result backend (db 2). **MinIO** es el object storage S3-compatible donde viven los documentos de las bases de conocimiento y los artefactos binarios.

Vault es la única vía de credenciales de la plataforma — claves de proveedores LLM, credenciales git de proyectos, secretos de MCP y SSO. Nada de esto toca la base de datos ni los logs: la interfaz solo sabe si una credencial «está configurada». El contenedor recupera la capability `IPC_LOCK` (sobre el baseline cap-drop ALL) para bloquear su memoria y que los secretos no acaben en swap. Todos estos servicios persisten en volúmenes nombrados que entran en el backup diario.



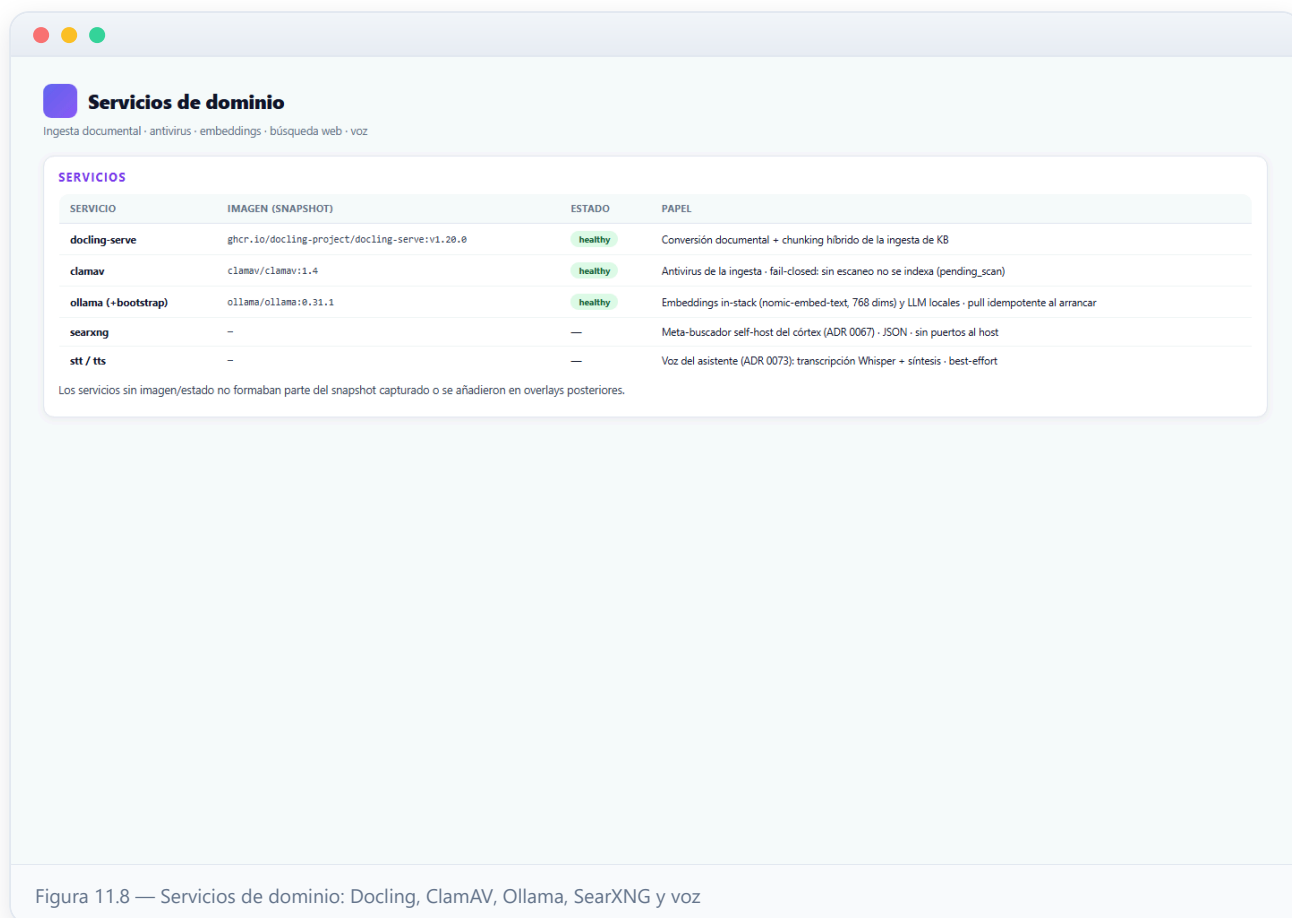
8 Servicios de dominio: Docling, ClamAV, Ollama, SearXNG y voz

docling-serve (IBM Docling) es el conversor documental de la ingesta: transforma PDF, Office y HTML en estructura y realiza el *chunking* híbrido con el que las bases de conocimiento se trocean antes de indexarse para el RAG.

ClamAV escanea cada documento subido ANTES de indexarlo, con política **fail-closed** (ADR 0105): si el antivirus no está disponible, el documento queda en estado `pending_scan` y se avisa al operador — nunca se indexa contenido sin escanear.

Ollama sirve los embeddings in-stack (por defecto `nommic-embed-text`, 768 dimensiones) y, opcionalmente, modelos LLM locales; su compañero one-shot **ollama-bootstrap** hace el `pull` idempotente del modelo de embeddings al levantar el stack, y el overlay GPU (`docker-compose.gpu.yml`) le añade aceleración CUDA si el host la tiene. Es el único servicio con límites de recursos ampliados (4 CPU / 8 GiB) junto a Docling.

SearXNG es el meta-buscador self-hosted que da búsqueda web al córtex (ADR 0067) sin depender de una API key externa; sirve JSON por configuración montada en solo lectura y no escucha en el host. Los servicios de **voz** (`stt` con Whisper y `tts`) dan el modo de voz del asistente (ADR 0073): descargan su modelo en el primer arranque (cache en volumen) y el canal de voz es best-effort mientras no estén listos.



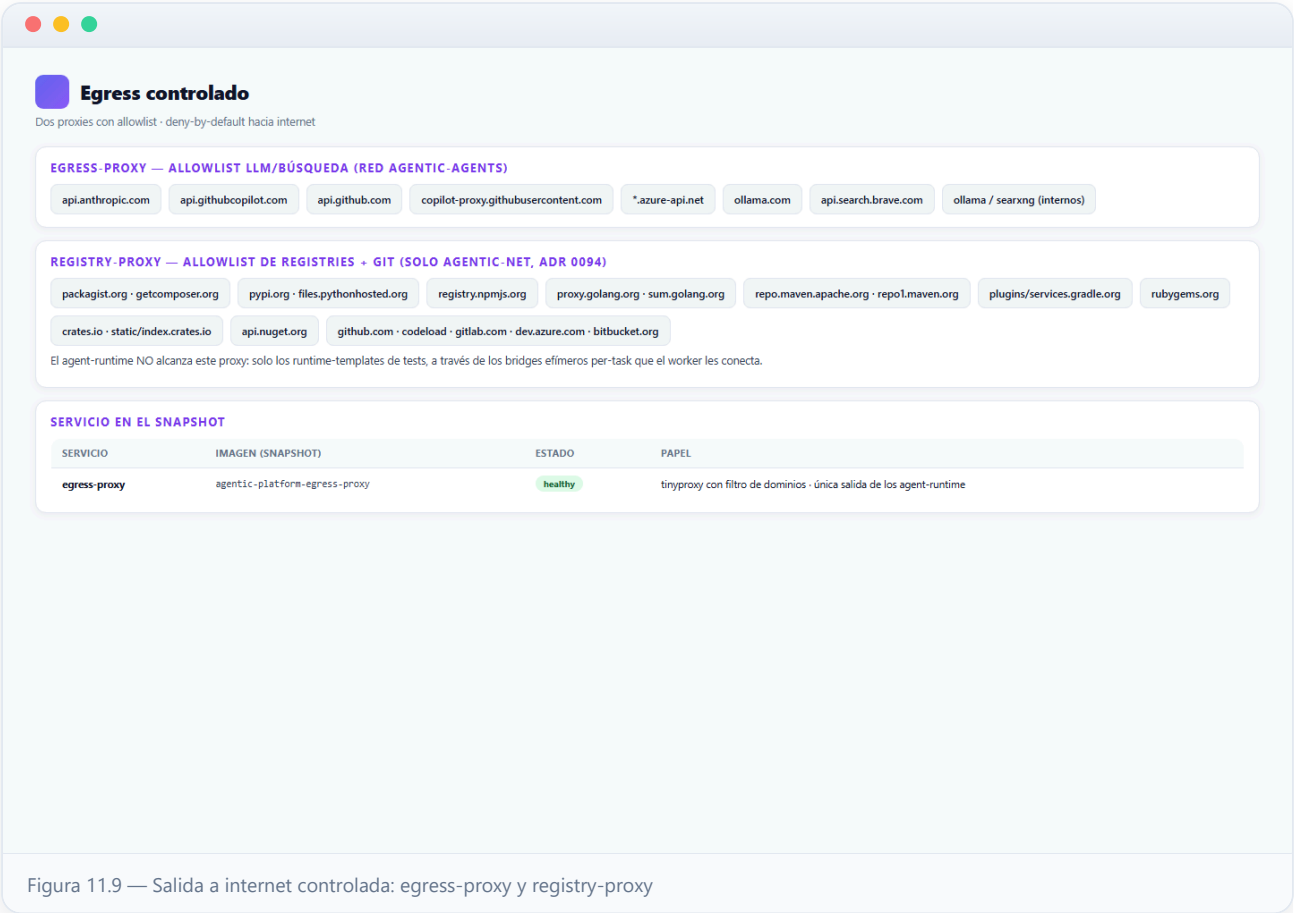
9 Salida a internet controlada: egress-proxy y registry-proxy

Los contenedores de agente viven en una red **interna** sin NAT: no pueden abrir conexiones a internet. Su única vía de salida es el **egress-proxy** (tinyproxy) con una **allowlist de dominios** limitada a los proveedores LLM y a la búsqueda:

`api.anthropic.com` , `api.githubcopilot.com` , `api.github.com` , `copilot-proxy.githubusercontent.com` , `*.azure-api.net` , `ollama.com` , `api.search.brave.com` y los servicios internos `ollama` / `searxng` . Las credenciales LLM NO pasan por el proxy (entran al runtime por `/run/secrets`): el proxy solo decide *qué hosts* se pueden alcanzar.

El **registry-proxy** (ADR 0094) es una **segunda instancia disjunta** de tinyproxy para los runtime-templates de tests: su allowlist cubre los registries de paquetes (Composer/Packagist, PyPI, npm, Go proxy, Maven/Gradle, RubyGems, crates.io, NuGet) y los hosts git (github.com, gitlab.com, dev.azure.com, bitbucket.org). La clave de seguridad es dónde vive: SOLO en `agentic-net` — el agent-runtime no puede alcanzarlo, así que el agente no descarga nada de PyPI o GitHub directamente. Es el worker quien conecta este proxy a los **bridges efimeros per-task** de cada runtime-template, siempre internos y sin NAT crudo.

Ambos proxies corren con el hardening estándar del stack (cap-drop ALL más las capabilities mínimas que tinyproxy necesita para degradar a su usuario de servicio, no-new-privileges) y con healthcheck propio.



Las redes del stack: agentic-net, agentic-agents y agentic-docker

El stack define tres redes con papeles nítidos. **agentic-net** es la red de plataforma (bridge estándar): en ella conviven Caddy, api-server, admin-panel, los datos (PostgreSQL, Redis, MinIO, Vault), los servicios de dominio (ClamAV, Docling, Ollama, SearXNG), los dos proxies de salida y la pila de monitorización.

agentic-agents es la red de los agentes y es **interna** (`internal: true`): Docker no le da NAT, así que ningún contenedor conectado SOLO a ella puede salir a internet. Ahí viven los **agent-runtime efímeros** y los **review-runtime**, junto al egress-proxy (su única salida), el api-server (callbacks internos y proxy de review) y los workers (que los orquestan). La comunicación entre contenedores está habilitada — el aislamiento fuerte del sandbox no se fía de la red, sino del perfil endurecido de cada runtime (cap-drop, seccomp, sin socket).

agentic-docker es la red interna más pequeña y estricta: SOLO los workers y el docker-socket-proxy se conectan a ella; es el canal exclusivo por el que se lanzan contenedores. Por último, para cada tarea de tests el worker crea un **bridge efímero per-task** (también interno) que une el runtime-template con el registry-proxy — y lo destruye al terminar; un reaper de fondo recoge los contenedores y redes huérfanos.

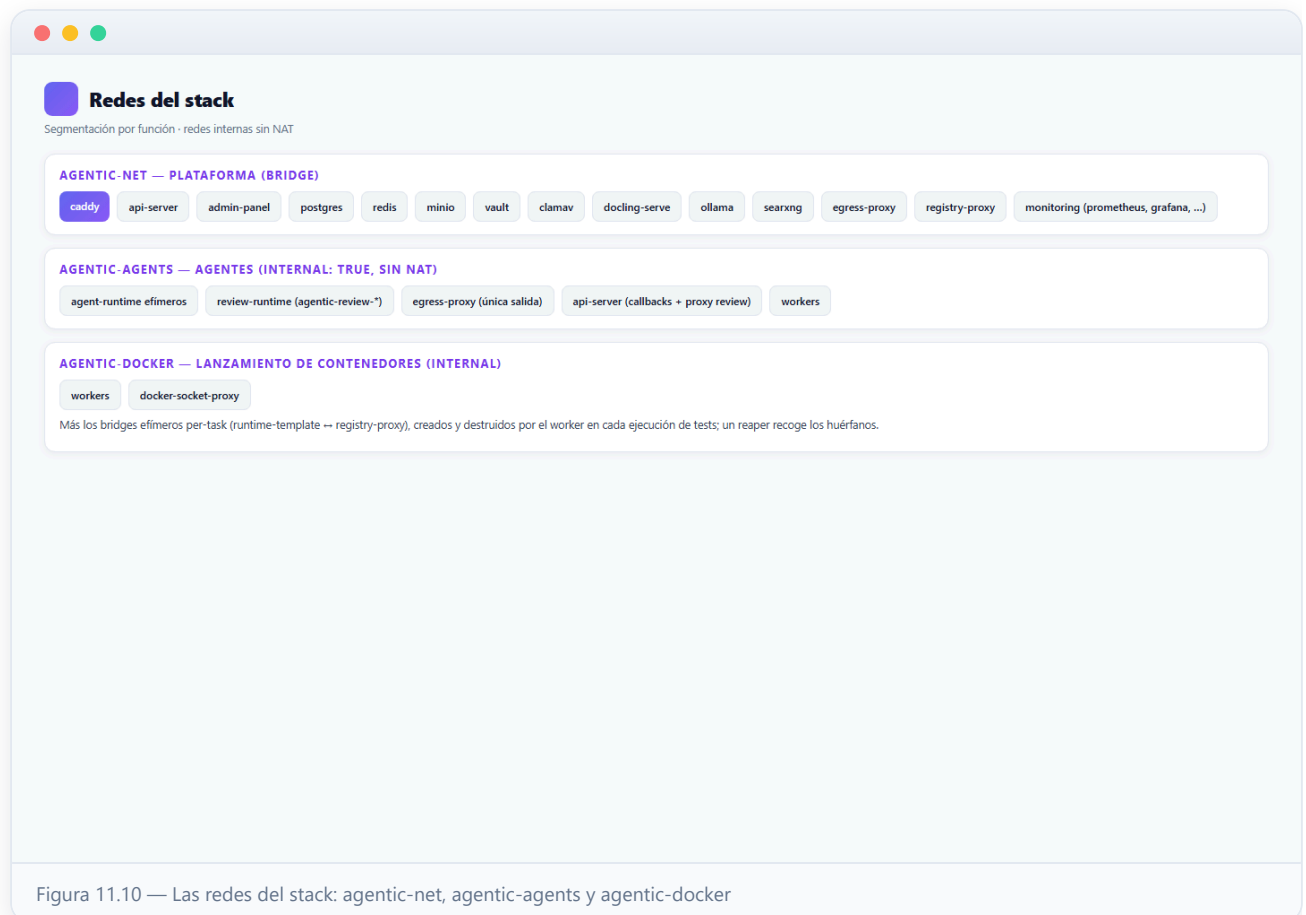


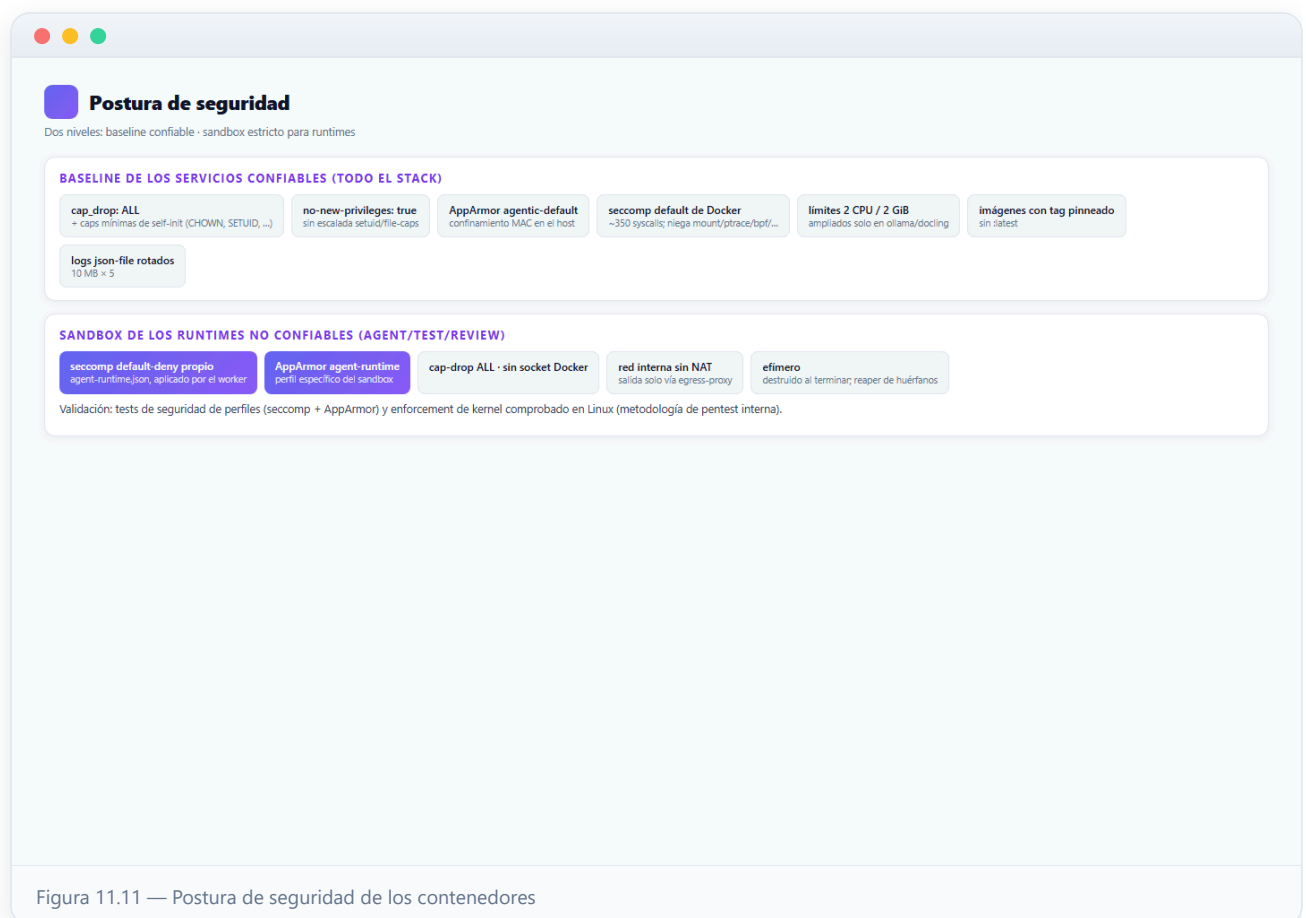
Figura 11.10 — Las redes del stack: agentic-net, agentic-agents y agentic-docker

11 Postura de seguridad de los contenedores

El modelo de amenaza distingue dos niveles. Los **servicios confiables** (imágenes oficiales y first-party de larga vida) llevan un baseline uniforme: `cap_drop: ALL` con una re-adición mínima de capabilities de auto-inicialización (CHOWN, DAC_OVERRIDE, FOWNER, SETGID, SETUID — lo justo para que postgres/redis/clamav preparen su directorio de datos y degraden a su usuario de servicio; Vault suma IPC_LOCK y SETFCAP), `no-new-privileges:true` (imposible escalar por setuid/file-caps), el perfil **AppArmor** `agentic-default` cargado en el host, y el **seccomp por defecto de Docker** — la allowlist de ~350 syscalls probada en batalla que ya niega mount, ptrace, kexec, bpf o init_module. Un perfil default-deny artesanal se probó aquí y rompía los servicios Go y postgres: para servicios confiables era un negativo neto.

El perfil **estricto** se reserva para donde importa: los **runtimes no confiables** (agent-runtime, test-runtime, review-runtime) reciben en el lanzamiento un seccomp **default-deny** propio (`agent-runtime.json`) y su perfil AppArmor específico, además de cap-drop ALL, ninguna vía al socket Docker y red interna sin NAT. El worker aplica estos perfiles al crear cada contenedor y lo destruye al terminar; su cumplimiento está validado por tests de seguridad y comprobado con enforcement de kernel en Linux.

Completan la postura: **límites de recursos** por defecto (2 CPU / 2 GiB por servicio, ampliados solo donde se justifica — Ollama, Docling) para que un contenedor desbocado no agote la máquina única; imágenes **pinneas a tag fijo** (nunca :latest) por reproducibilidad y cadena de suministro; y rotación de logs json-file (10 MB × 5 ficheros) en todos los servicios.

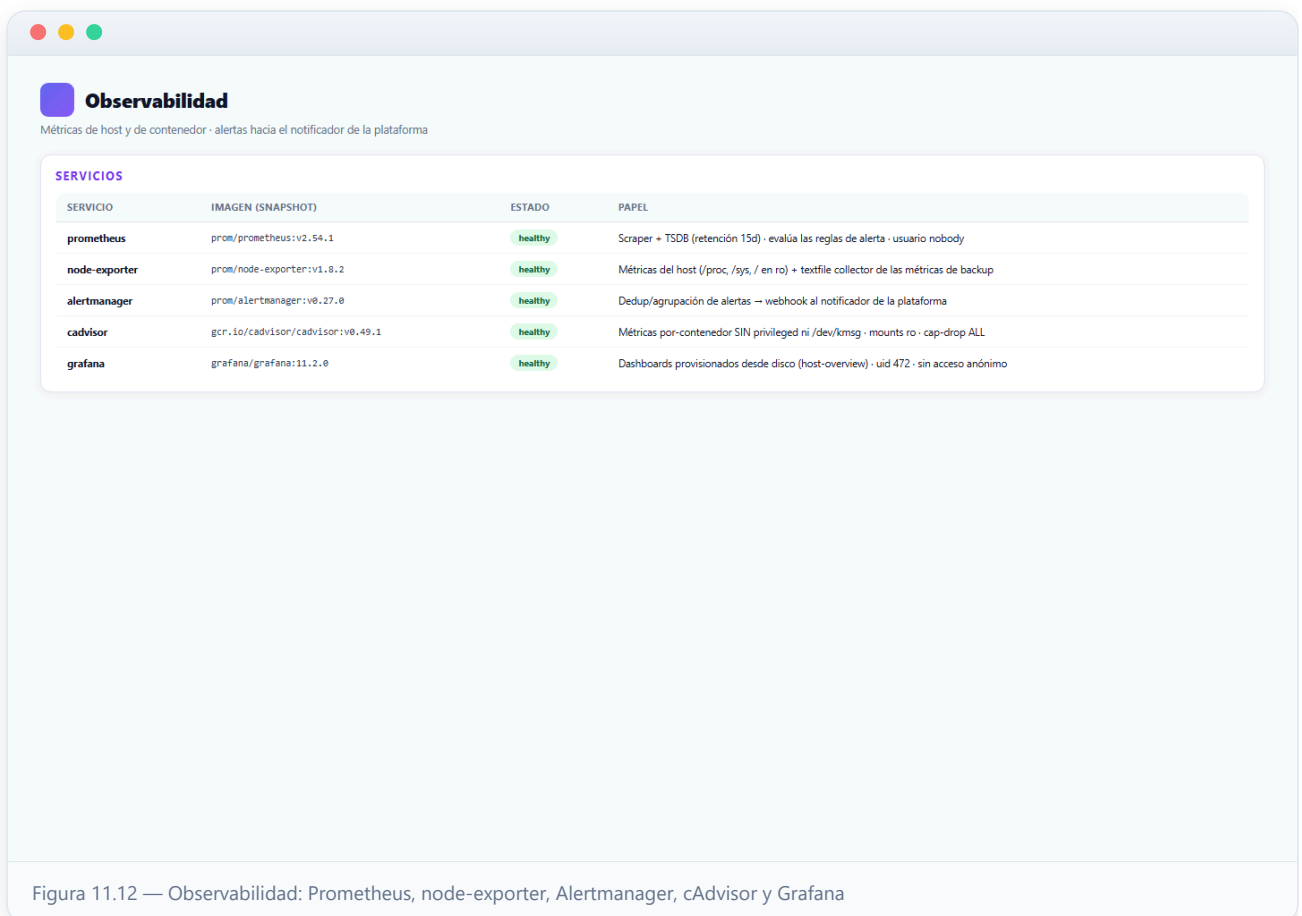


Observabilidad: Prometheus, node-exporter, Alertmanager, cAdvisor y Grafana

El overlay de monitorización añade el plano de métricas y alertas. **Prometheus** es el scraper y la base de series temporales (retención de 15 días): recoge las métricas de node-exporter y cAdvisor y evalúa las reglas de alerta de host; corre como usuario *nobody* con su configuración montada en solo lectura. **node-exporter** exporta las métricas del host (CPU, RAM, disco, swap, red) montando `/proc`, `/sys` y `/` en solo lectura, y además re-exporta por su *textfile collector* las métricas que el motor de backup escribe tras cada ejecución — la fuente de la alerta de «último backup fallido».

Alertmanager recibe las alertas que disparan las reglas, las dedup-agrupa y las entrega por webhook al notificador de la plataforma, de modo que acaban en el inbox de notificaciones del panel. **cAdvisor** aporta las métricas por-contenedor (CPU, memoria, red, filesystem) y está endurecido sin `privileged` ni `/dev/kmsg`: obtiene los stats de bind-mounts de solo lectura sobre el estado de Docker y cgroups, con `cap-drop ALL` y `no-new-privileges`. El trade-off consciente es que pierde la decodificación de OOM-kills del kernel; el runbook de monitorización documenta el override legacy-privileged como opt-in para el host que lo necesite.

Grafana sirve los dashboards *provisionados desde disco* (datasource y dashboard host-overview: CPU/RAM/disco/red + métricas por contenedor) — los ficheros del repo son la fuente de verdad, sin clics manuales. Corre con su uid no-root (472), sin registro de usuarios ni acceso anónimo ni telemetría, y con la contraseña de administración inyectada por entorno/Vault.

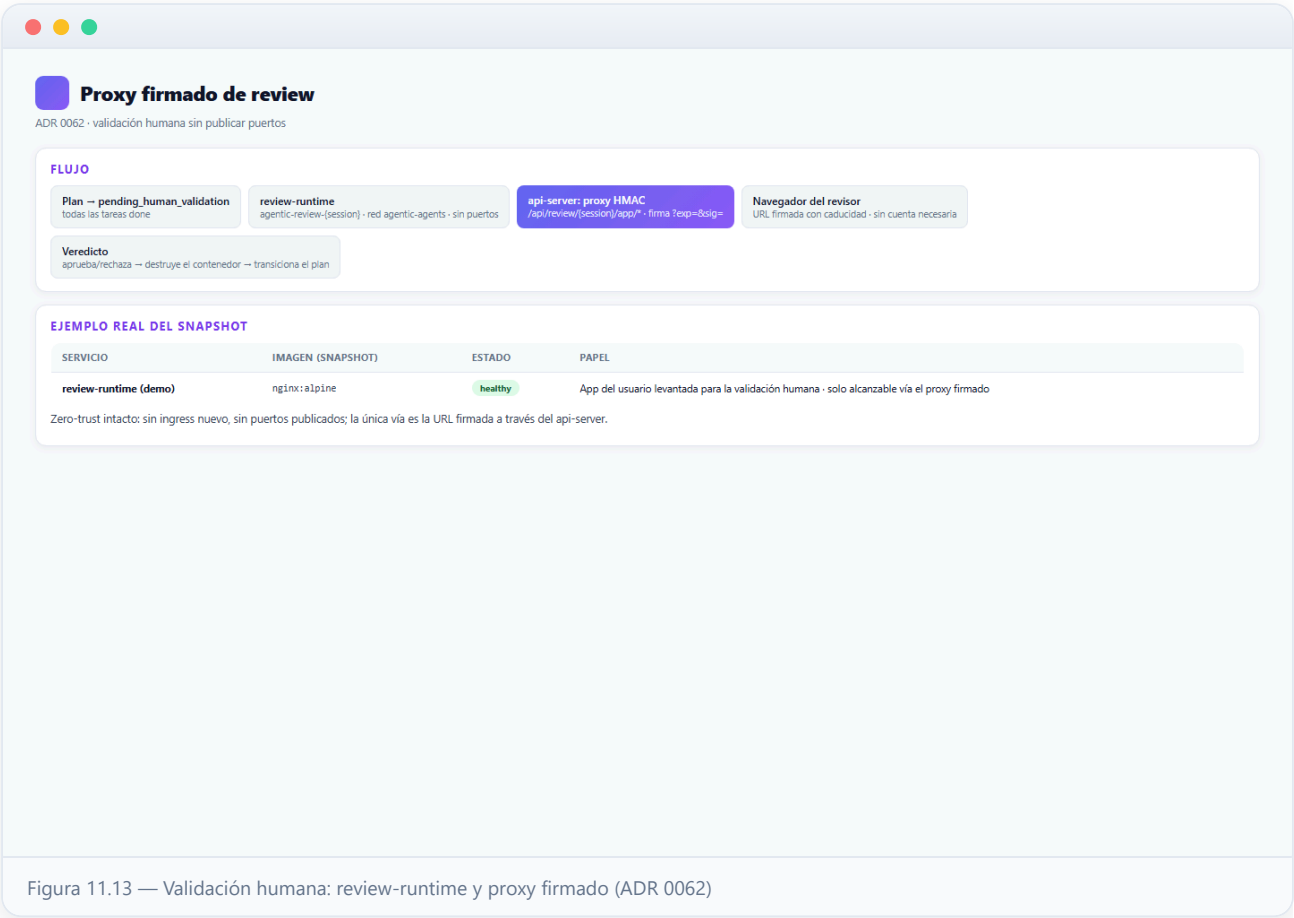


13 Validación humana: review-runtime y proxy firmado (ADR 0062)

Cuando todas las tareas de un plan terminan, el plan pasa a *pending_human_validation* y el sistema levanta un **review-runtime**: un contenedor persistente con la app del usuario servida en su puerto principal y el worktree del plan montado, para que un humano la abra en su navegador y la pruebe antes de aprobar o rechazar.

El problema que resuelve el ADR 0062 es que la red es zero-trust: ese contenedor vive en `agentic-agents` (interna), sin puertos al host y sin ruta estática en Caddy. La solución es que **el api-server actúa de proxy inverso firmado**: la ruta `/api/review/{session}/app/...` reenvía cada petición al contenedor de review — direccionado por su nombre determinista (`agentic-review-{session}`) — y la autenticación es una **firma HMAC en la URL** (`?exp=&sig=`) con caducidad. No hace falta JWT: el revisor puede no tener cuenta en la plataforma, el enlace firmado ES la credencial, y la app jamás se publica directamente.

El panel del operador obtiene un **enlace clicable recién firmado** («Abrir app para probar») a través de un endpoint protegido con JWT+RBAC. Al emitir el **veredicto** (aprobar/rechazar), la sesión se marca terminal, el contenedor se **destruye** y el plan transiciona a completado o rechazado. En el snapshot de este manual puede verse un contenedor real de demostración (`agentic-review-demo`) de este mecanismo.



14 Copias de seguridad y durabilidad de los datos

El **backup diario** (03:00 por defecto, cron configurable desde el panel — manual 09) lo ejecuta el pool dedicado **workers-backup** por la cola `privileged`, con un único slot de concurrencia. Es el único pool que corre como root dentro de su contenedor: necesita leer los datos internos de Redis y de Vault (directorios 0700 de otros uids) para empaquetarlos, y un restore además escribe en ellos. Ese pool no ejecuta nunca runs de agentes.

Cada bundle incluye el **pg_dump** completo de PostgreSQL, el **tar** de los volúmenes de MinIO, Redis y Vault y el **data-root de agentes** (bare repos y worktrees), más su manifest. Tras verificarse, el bundle se sube a los **destinos remotos** configurados (S3, Backblaze B2, SFTP/NAS, rclone) con credenciales resueltas del secret seam (Vault/entorno — nunca de la base de datos), y la retención local elimina los bundles antiguos tras un backup correcto.

La durabilidad se refuerza en dos frentes: los bundles se escriben en un **bind del host** fuera de los volúmenes de Docker (sobreviven incluso a perder el engine entero) y el data-root de agentes es un volumen nombrado **externo** que ni `docker compose down -v` elimina. El resultado de cada ejecución se publica como métrica en el textfile collector de node-exporter, de donde salen las alertas de «último backup fallido» y «backup demasiado antiguo» que llegan al notificador de la plataforma.

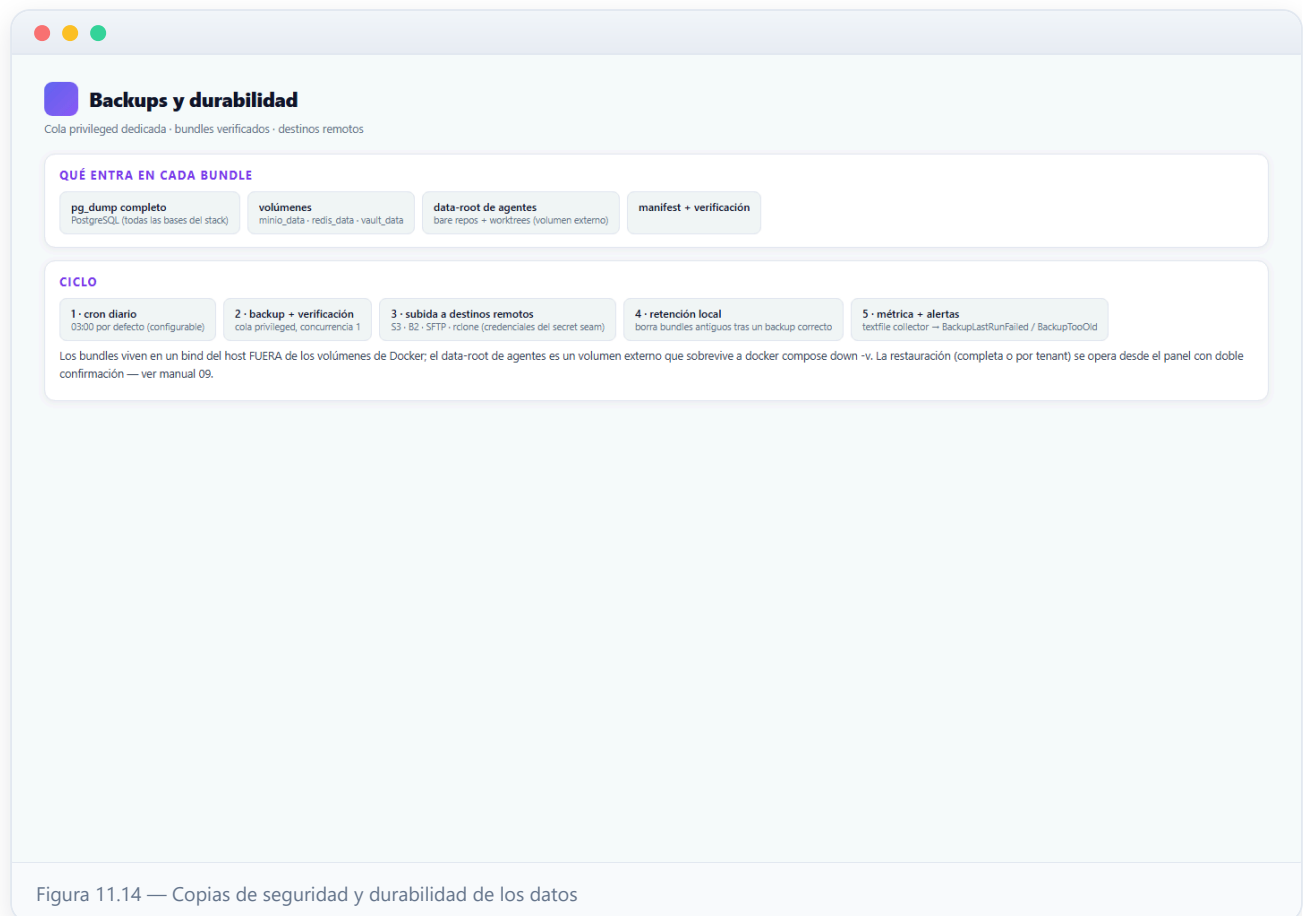


Figura 11.14 — Copias de seguridad y durabilidad de los datos